Практическая работа № 2.
Реализация полносвязных нейронных сетей с использованием программных библиотек

Цель работы – научиться строить, обучать и анализировать полносвязные нейронные сети (Multi-Layer Perceptron, MLP) с использованием фреймворка PyTorch, понимая математическую суть операций, скрывающихся за программным интерфейсом.

Задачи:
1. Освоить базовый API модуля torch.nn для создания архитектур сетей.
2. Реализовать полный цикл обучения, понимая роль каждого шага с точки зрения математической оптимизации.
3. Научиться работать с различными функциями потерь и оптимизаторами.
4. Исследовать влияние гиперпараметров (глубина сети, функции активации) на сходимость и обобщающую способность модели.

1. Теоретическая справка

Полносвязная нейронная сеть (MLP) аппроксимирует отображение

$$f: \mathbb{R}^d \to \mathbb{R}^k$$

с помощью суперпозиции аффинных преобразований и нелинейных функций активации. Для L-слойной сети выход l-го слоя вычисляется как:

$$\mathbf{h}^{(l)} = \sigma_l\left(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}\right), \quad l = 1, \dots, L,$$

где $\mathbf{h}^{(0)} = \mathbf{x}$ — входной вектор, $\mathbf{W}^{(l)}$ — матрица весов, $\mathbf{b}^{(l)}$ — вектор смещений, σ_l — функция активации.

Обучение сети сводится к задаче безусловной минимизации эмпирического риска:

$$\min_{\theta} \mathcal{L}(\theta) = \frac{1}{N}\sum_{i=1}^{N} \ell\left(f(\mathbf{x}_i; \theta), y_i\right)$$

где $\theta = \{\mathbf{W}^{(l)}, \mathbf{b}^{(l)}\}_{l=1}^{L}$ — параметры модели, ℓ — функция потерь (Loss).

Фреймворк PyTorch использует механизм *автоматического дифференцирования* (Autograd). При прямом прохождении (forward pass) строится вычислительный граф. При вызове .backward() применяется цепное правило (chain rule) для вычисления градиентов $\nabla_\theta \mathcal{L}$, после чего оптимизатор обновляет параметры: $\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}(\theta_t)$.

2. Разбор типовых задач

Задача 1. Базовый цикл обучения для задачи регрессии

Задание: Обучить MLP для аппроксимации функции $y = \sin(x) + \epsilon$, где $\epsilon \sim \mathcal{N}(0, 0.1)$.
Математическая постановка: Минимизация MSE: $\mathcal{L} = \frac{1}{N}\sum(y_i - \hat{y}_i)^2$.

```
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np

# 1. Подготовка данных (тензоры)
X = torch.linspace(-np.pi, np.pi, 1000).reshape(-1, 1)
y = torch.sin(X) + 0.1 * torch.randn_like(X)
```

```
# 2. Определение архитектуры MLP
class SimpleMLP(nn.Module):
    def __init__(self):
        super().__init__()
        # Математически: h1 = ReLU(W1*x + b1), h2 = ReLU(W2*h1 + b2), out = W3*h2 + b3
        self.net = nn.Sequential(
            nn.Linear(1, 32),
            nn.ReLU(),
            nn.Linear(32, 32),
            nn.ReLU(),
            nn.Linear(32, 1)
        )
    def forward(self, x):
        return self.net(x)
model = SimpleMLP()

# 3. Определение функции потерь и оптимизатора
criterion = nn.MSELoss() # Математически: mean((y - y_hat)^2)
optimizer = optim.Adam(model.parameters(), lr=1e-3)

# 4. Цикл обучения
epochs = 200
for epoch in range(epochs):
    # Forward pass: вычисление предсказаний и построение графа
    predictions = model(X)
    loss = criterion(predictions, y)
    # Backward pass: вычисление градиентов (цепное правило)
    optimizer.zero_grad() # Обнуление накопленных градиентов!
    loss.backward()
    # Optimization step: обновление весов
    optimizer.step()
    if (epoch + 1) % 50 == 0:
        print(f'Epoch [{epoch+1}/{epochs}], Loss: {loss.item():.4f}')
```

Задача 2. Классификация и численная стабильность

Задание: Решить задачу многоклассовой классификации.

В PyTorch `nn.CrossEntropyLoss` **уже включает** в себя `LogSoftmax`. Типичная ошибка: применение `nn.Softmax` на выходе сети, а затем передача в `nn.CrossEntropyLoss`. Это приведет к потере численной стабильности и некорректным градиентам.

```
# Правильный подход:
# Выходной слой НЕ имеет активации (возвращает "логиты" - сырые оценки)
model = nn.Sequential(nn.Linear(10, 5)) # 10 входов, 5 классов
criterion = nn.CrossEntropyLoss() # Внутри: LogSoftmax + NLLLoss
# optimizer = ...
# logits = model(X_batch)
# loss = criterion(logits, y_batch) # y_batch содержит индексы классов (0, 1, 2
, 3, 4)
```

3. Задачи для самостоятельного разбора на занятии

Задача 1 (Математическая разминка).

Дана сеть: `Input(784) -> Linear(256) -> ReLU -> Linear(128) -> ReLU -> Linear(10)`.

1. Вычислите точное количество обучаемых параметров в этой сети.

2. Сколько операций умножения (FLOPs) требуется для одного прямого прохода (forward pass) одного объекта batch'а (без учета смещений)?
Подсказка: Параметры линейного слоя `Linear(in, out)` *— это* $in \times out + out$.

Задача 2 (Перевод с языка математики на язык кода).

Запишите на PyTorch архитектуру сети, которая математически описывается следующим образом:

$$\mathbf{h}_1 = \tanh(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1)$$

$$\mathbf{h}_2 = \mathbf{h}_1 \odot \sigma(\mathbf{W}_2\mathbf{h}_1 + \mathbf{b}_2)$$

$$\mathbf{y} = \mathbf{W}_3\mathbf{h}_2 + \mathbf{b}_3$$

где $\odot$ — поэлементное умножение, σ — сигмоида.

Указание: Используйте `nn.Module` *и переопределите метод* `forward`*, так как* `nn.Sequential` *не поддерживает такие skip-connections.*

Задача 3 (Поиск и исправление ошибок).

В предоставленном коде студент пытается обучить сеть, но loss не уменьшается, а accuracy стоит на уровне случайного угадывания. Найдите **три** критические ошибки.

```
model = nn.Sequential(nn.Linear(20, 50), nn.Softmax(dim=1), nn.Linear(50, 10))
criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(model.parameters(), lr=0.01)

for epoch in range(10):
    for X_batch, y_batch in dataloader:
        out = model(X_batch)
        loss = criterion(out, y_batch)
        loss.backward()
        optimizer.step()
```

Задача 4 (Функция активации).

Реализуйте функцию активации **Swish** ($f(x) = x \cdot \sigma(x)$, где σ — сигмоида) в виде собственного класса, наследуемого от `nn.Module`. Убедитесь с помощью `torch.autograd.gradcheck`, что аналитический градиент, вычисляемый PyTorch, совпадает с численным.

Задача 5 (Работа с данными).

Создайте кастомный класс `Dataset`, наследуемый от `torch.utils.data.Dataset`, который генерирует пары (x, y), где $x \sim \mathcal{U}(-2,2)$, а $y = x^3 - 2x$. Оберните его в `DataLoader` с `batch_size=32` и `shuffle=True`.

Задача 6 (Исследование оптимизаторов).

Обучите глубокую сеть (5 слоев по 64 нейрона, активация ReLU) на сложной невыпуклой функции (например, $y = \sin(5x) \cdot e^{-x^2}$). Постройте график зависимости Loss от Epoch для трех оптимизаторов: `SGD` (без momentum), `SGD` (с momentum=0.9) и `Adam`. Сделайте вывод о скорости сходимости.

4. Задачи для домашнего решения

1. **Ручная реализация Weight Decay.** Реализуйте L_2-регуляризацию *вручную*, добавив к функции потерь штраф $\frac{\lambda}{2}\sum\|\mathbf{W}\|_2^2$ (смещения не штрафуем). Сравните скорость обучения и итоговый loss с использованием встроенного параметра `weight_decay` в оптимизаторе `AdamW`. Почему для Adam рекомендуется использовать именно `AdamW`, а не обычный `Adam` с `weight_decay`?
2. **Линейный автоэнкодер и PCA.** *Математическая часть:* Докажите, что обучение линейного автоэнкодера (входной слой -> сжатие до k нейронов -> восстановление до исходной размерности) с MSE-функцией потерь эквивалентно нахождению первых k главных компонент (PCA). *Практическая часть:* Обучите такой автоэнкодер в PyTorch на датасете MNIST (без нелинейностей!). Извлеките веса первого слоя и сравните их с результатом работы `sklearn.decomposition.PCA`. Визуализируйте полученные "компоненты" как изображения.
3. **Ручная реализация Dropout.** Реализуйте слой Dropout в режиме обучения и тестирования как собственный `nn.Module`. В режиме обучения умножайте активации на маску Бернулли $m \sim \text{Bernoulli}(p)$ и делите на $(1-p)$ (Inverted Dropout). В режиме тестирования слой должен быть тождественным преобразованием. Сравните кривые обучения сети с вашим слоем и с `nn.Dropout`.
4. **Анализ затухания градиентов.** Создайте очень глубокую сеть (20 скрытых слоев по 64 нейрона). Попробуйте обучить ее с активациями `Sigmoid` и `ReLU`. Для каждой сети выведите норму градиентов первого слоя (`model[0].weight.grad.norm()`) на первой итерации. Объясните разницу с точки зрения производных функций активации.

5. Вопросы для обсуждения (семинарская часть)

1. Почему нейронным сетям нужны нелинейные функции активации? Что будет, если все слои MLP будут линейными (`nn.Linear` без активации)?
2. В чем разница между `nn.CrossEntropyLoss` и `nn.NLLLoss`? Почему первая требует "сырых" логитов, а вторая — логарифмов вероятностей?
3. Зачем нужен `optimizer.zero_grad()`? Что произойдет с вычислительным графом и градиентами, если убрать эту строку?
4. Как PyTorch понимает, какие переменные нужно дифференцировать? Роль атрибута `requires_grad=True` и концепция "вычислительного графа".
5. Почему для `ReLU` инициализация He (Kaiming) работает лучше, чем Xavier (Glorot)?

6. Критерии оценивания

Критерии оценивания работы на занятии

Баллы	Описание
Отлично (9-10)	Студент активно участвует, самостоятельно решает все аудиторные задачи, может объяснить математический смысл каждого шага кода, успешно проходит устную защиту.
Хорошо (7-8)	Студент решает большинство задач, допускает minor-ошибки, которые исправляет с минимальной подсказкой. Понимает связь кода и математики.
Удовлетворительно (5-6)	Студент решает только базовые задачи (1-3). Требуется помощь преподавателя для отладки кода. Понимание математической сути поверхностное.
Неудовлетворительно (<5)	Студент не приступал к выполнению заданий или не может объяснить написанный им же код.

Итоговая формула:

- аудиторная работа (решение задач 1-6) — **40%**
- домашнее задание (задачи 1-4) — **40%**
- устный опрос / Защита отчета — **20%**

Чек-лист: что я должен уметь после изучения темы

- Я могу перевести математическое описание архитектуры MLP в код на PyTorch.
- Я понимаю, что происходит “под капотом” при вызове `.backward()`.
- Я знаю, почему `CrossEntropyLoss` в PyTorch не требует `Softmax` на выходе.
- Я умею создавать слои и функции потерь, наследуя `nn.Module`.
- Я могу отследить и исправить проблемы затухающих/взрывающихся градиентов.
- Я понимаю разницу между параметрами модели и гиперпараметрами обучения.